SCIM211 SIMULATION MODELLING · GROUP MINI PROJECT · 9%

Café Poisson: SCIM211 Coffee Shop Simulation Hackathon

Project instructions, model rules, readiness check, hackathon day, and marking.

Group Mini Project: 9%

The Story

Café Poisson has just opened next to campus. The owner named it after the Poisson process, because the customers really do seem to arrive at random.

A normal lunch at Café Poisson

The café is open all day, but lunch is the rush. From 12:00 to 14:00 about 30 customers come in every hour, but never on a timetable: for a few minutes nobody comes, then three people walk in at once. At 14:00 the café closes its door for one hour so that the staff can eat their own lunch, clean, and restock, and then it opens again for the afternoon. **Your model covers the lunch session, 12:00 to 14:00.**

Two baristas work behind the counter. Each makes one drink at a time. Everyone waits in one line, and the person at the front goes to whichever barista is free first.

The menu has two kinds of drink. **Classic drinks** (Americano, latte, cappuccino, Thai tea, cocoa) are quick, about 2 minutes each. **Specialty drinks** (matcha latte, coffee frappé, mango smoothie, cocoa frappé) are blended or need extra steps, so they take about 4 minutes, but they sell for more. A little under half of the customers (45%) order a classic drink.

Customers are patient, but not forever. Each customer will wait somewhere between 5 and 15 minutes for their drink to be started; after that they give up and walk out. The shop also has room for only 8 people to stand in line. If the line is full when someone arrives, they take one look and leave.

Then there is the money. A classic drink sells for **฿85** and its ingredients cost **฿25**, so the café keeps ฿60. A specialty drink sells for **฿120** and its ingredients cost **฿40**, so the café keeps ฿80. Each barista costs the café **฿120 an hour**, paid for the full two hours whether they are busy or not: ฿480 a day for the two baristas. A customer who leaves without a drink pays nothing.

Café Poisson at a glance

The menu

	Classic drinks	Specialty drinks	In the code
Examples	Americano, latte, cappuccino, Thai tea, cocoa	matcha latte, coffee frappé, mango smoothie, cocoa frappé	`simple` / `complex`
Price	฿85	฿120	`price_simple`, `price_complex`
Ingredient cost	฿25	฿40	`cost_simple`, `cost_complex`

	Classic drinks	Specialty drinks	In the code
Average time to make	2 minutes	4 minutes	`mean_service_simple`, `mean_service_complex`
Share of customers	45%	55%	`simple_order_probability` = 0.45

The café on a normal lunch

	A normal lunch	In the code
Lunch session (what you model)	12:00–14:00, 120 minutes (minute 0 is 12:00); the door then closes for the staff break	`run_minutes` = 120
Customers	about 30 per hour, arriving at random	`arrival_rate_per_hour` = 30
Baristas	2	`n_baristas` = 2
Barista cost	฿120 per barista per hour, for the full 120 minutes	`barista_cost_per_hour` = 120
Patience	between 5 and 15 minutes (10 on average)	`mean_patience` = 10
Room to wait	8 people in line	`max_queue` = 8

The exact rules for your model are in the Model Rules section below.

The owner's questions

Every week the owner has a new question. "Should I hire another barista for the lunch rush?" "Is a matcha promotion worth it?" "What should I do when the coffee machine slows down?" These ideas cannot be tested on real customers, because a bad idea costs real money and real customers.

So the owner has hired your group. Your job is to build a **simulation of Café Poisson**: a Python model of the café that can be run again and again under different plans.

In the Week 16 Wednesday lab, the owner will send each group one urgent question that you have not seen before. You will have one hour to adapt your model, run an experiment, and decide what the café should do. In the second hour, you will pitch your advice to the owner (and to the class).

You do not need a fancy café. You need a model that is correct, easy to change, reproducible (the same seed gives the same result), and good enough to make a decision with.

You work in groups of three. At the end, each of you shares 15 points among your group to show who did what (see below).

Assessment

Component	Marks
Prepared and reusable café simulation	4
Live challenge implementation and experiment	3
Pitch communication	1
Individual oral understanding	1
Total	**9**

Working as a Group of Three

You work in a group of **three** for the whole project. Every member must understand the code and be ready to explain it.

At the end of the project, each member privately shares **15 points** among all three members of the group, **including themselves**, using a **Google Form**. The link will be posted in Google Classroom.

- If everyone worked equally, give each member **5** points.
- If not, move points to show who did more or less. Use at most one decimal place, for example **5.1, 4.9, 5.0**. No member can get a negative number.
- Your points should add up to **15**. A total below 15 is accepted.
- A total **above 15** is a penalty: every point above 15 is taken off your own project mark. For example, if your points add up to 15.5, you lose 0.5 marks.
- If you do not hand in your points, they are counted as 5, 5 and 5.

Each member's share of the group marks is then weighted according to the points they receive from the group.

What Your Café Model Must Include

Your prepared model must have:

1. a 120-minute lunch session, with time measured in minutes
2. customers who arrive at random
3. two kinds of order: classic drinks and specialty drinks
4. random service times that depend on the order
5. a number of baristas that can be changed
6. one queue, served first come, first served
7. impatient customers who leave the queue if they wait too long
8. events for arrival, start of service, end of service, and running out of patience
9. a system state: the clock, the waiting customers, and which baristas are free
10. a maximum queue length that can be changed, with turned-away customers counted as lost
11. revenue, ingredient cost, staff cost, and profit
12. random seeds, so that any result can be reproduced
13. repeated runs over a list of seeds
14. all the outputs listed below.

Model Rules

Every group uses the same rules. This lets the checker test your numbers, and lets us compare the groups' results during the pitches.

Part of the model	Rule
Arrivals	A Poisson process with rate `arrival_rate_per_hour`. The gaps between arrivals are independent and exponential with mean `60 / arrival_rate_per_hour` minutes. Customers arrive only during `[0, run_minutes)`. If the rate is 0, nobody arrives.

Part of the model	Rule
Order type	Each customer orders a classic (simple) drink with probability `simple_order_probability`, otherwise a specialty (complex) drink, independently of everyone else.
Service time	Exponential, with mean `mean_service_simple` (classic) or `mean_service_complex` (specialty).
Patience	Each customer's patience is uniform between `0.5 * mean_patience` and `1.5 * mean_patience` minutes. If service has not **started** by `arrival time + patience`, the customer leaves the queue (**reneges**) and is lost. Once service starts, patience no longer matters. If `mean_patience` is `None`, nobody reneges.
Queue limit	`max_queue` counts customers who are **waiting**, not those being served. If a customer arrives when every barista is busy and `max_queue` customers are already waiting, the customer leaves at once (**balks**) and is lost. If `max_queue` is `None`, there is no limit.
Queue order	One first-come, first-served queue feeds identical baristas. A free barista immediately serves the customer who has waited longest and is still in the queue.
End of the session	At `run_minutes` the door closes for the staff break, so no new customers arrive. Customers already accepted are allowed to finish. Customers still waiting may still renege after the door closes.

Keep the prepared `cafe_model.py` to these rules only. Do not add extras such as new customer types or breaks yet. The owner's question on hackathon day will ask you to change the rules.

Write your model so that inputs can be changed without rewriting the program. The owner's question may change the arrivals, the staff, the service times, who is served first, who may join the queue, the menu, a promotion, the equipment, or the end of the session.

What to Submit

Submit through Google Classroom by **17 November 2026, 23:59**. Submit one folder named with your group number:

```
GROUP_01/
├── cafe_model.py
├── model_notes.md
└── AI_USE.md
```

You may develop in a notebook, but the assessed model must run from `cafe_model.py`. The checker does not read notebooks.

`model_notes.md`

Write briefly:

- what real system the model represents
- your arrival and service-time assumptions
- how the queue and the baristas work
- what each output means, with its units
- at least three checks you used to test the model
- whether your model is a Markov chain, and why or why not: what state would you need, and which rule (if any) breaks the Markov property?

- one important limitation.

`AI_USE.md`

`AI_USE.md` is your group's **AI Use Acknowledgment Statement**, required by the course AI policy. Hand in **one per group**, in the same folder and by the same deadline as `cafe_model.py`. It covers everything you prepared before the hackathon. The first hour of the hackathon is AI-free, so it needs no statement.

Start from `AI_USE_template.md` and fill in five short sections:

1. **Tools used:** each AI tool (and version, if you know it). If you used none, write "We did not use any AI tool in this project." and stop there.
2. **What we used it for:** one line per use, saying which file and which part, for example "`cafe_model.py`, reneging: asked how to remove customers whose patience ran out from the queue".
3. **How we checked it:** what you did to make sure the output was correct, for example tracing three customers by hand or running the readiness checker.
4. **What we changed or rejected:** at least one example of AI output that was wrong, unclear, or not used, and what you did instead.
5. **Understanding:** confirm that every member can explain the AI-assisted parts, and say who checked each part.

In the oral questions, you may be asked to show and explain any part you list here.

Using AI responsibly

The course AI Collaboration Level 3 policy applies to the prepared model, `model_notes.md`, and `AI_USE.md`. AI may help you with ideas, explanations, drafts, or feedback on code, but your group must check and understand everything you submit. In the pitch, any member may be asked to explain any part of the code, including parts written with AI help.

The first hour of the hackathon is AI Level 1 (No AI); see Hackathon Day below.

Undeclared AI use, made-up results, or code that the group cannot explain are handled under the course academic-integrity policy.

The Python Interface

`cafe_model.py` must define these two functions, with exactly these names and arguments:

```
def simulate_cafe(config, seed):
    """Run one complete café replication and return a result dictionary."""


def run_replications(config, seeds):
    """Run one replication per seed and return a pandas DataFrame."""
```

Do not rename them. The file must run without keyboard input, a notebook, a menu, or any editing after you submit it.

Put your own experiments, printing, and plots at the bottom of the file under `if __name__ == "__main__":`, as in the template. That code runs when you run the file yourself, but not when the checker imports it.

You may add any other functions or classes you need.

Use **Python 3.10 or later** and only the standard library, **NumPy**, **pandas**, **SciPy**, and **Matplotlib**. These are the packages on the lab computers, and the hackathon hour is offline, so other packages (for example SimPy) will not be available.

The Configuration Dictionary

`simulate_cafe(config, seed)` must use every entry below:

Key	Meaning
`run_minutes`	Time during which new customers may arrive
`arrival_rate_per_hour`	Mean number of arrivals per hour
`n_baristas`	Number of identical baristas
`simple_order_probability`	Probability that a customer orders a classic (simple) drink
`mean_service_simple`	Mean service time for a classic drink, in minutes
`mean_service_complex`	Mean service time for a specialty drink, in minutes
`mean_patience`	Mean customer patience in minutes, or `None` if customers never leave the queue
`price_simple`	Price of a classic drink, in baht
`price_complex`	Price of a specialty drink, in baht
`cost_simple`	Ingredient cost of a classic drink, in baht
`cost_complex`	Ingredient cost of a specialty drink, in baht
`barista_cost_per_hour`	Staff cost per barista per hour, in baht
`max_queue`	Maximum number of customers waiting (not being served), or `None` for no limit

The checker uses its own values, so never hard-code these numbers inside the model.

This is Café Poisson on a normal day. Use it while you develop:

```
BASE_CONFIG = {
    "run_minutes": 120.0,
    "arrival_rate_per_hour": 30.0,
    "n_baristas": 2,
    "simple_order_probability": 0.45,
    "mean_service_simple": 2.0,
    "mean_service_complex": 4.0,
    "mean_patience": 10.0,
    "price_simple": 85.0,
    "price_complex": 120.0,
    "cost_simple": 25.0,
    "cost_complex": 40.0,
    "barista_cost_per_hour": 120.0,
    "max_queue": 8,
}
```

New customers arrive only before `run_minutes`, but accepted customers are allowed to finish. So `finish_time` can be later than `run_minutes`.

The Result Dictionary

`simulate_cafe(config, seed)` must return a dictionary with every field below:

Field	Meaning
`run_minutes`	Copy of `run_minutes` from the configuration
`finish_time`	The later of `run_minutes` (when the door closes) and the last service completion
`n_arrivals`	Number of customers who arrived
`n_served`	Number who finished service
`n_lost`	Number who left without being served: `n_balked + n_reneged`
`n_balked`	Number turned away on arrival because the queue was full
`n_reneged`	Number who left the queue when their patience ran out
`n_served_simple`	Classic (simple) orders completed
`n_served_complex`	Specialty (complex) orders completed
`mean_wait`	Mean waiting time of served customers, in minutes
`p90_wait`	90th percentile of served customers' waiting times. Use `np.percentile(waits, 90)` with NumPy's default (linear) method; other percentile definitions give different values with small samples and can fail the statistical checks.
`max_wait`	Longest waiting time of a served customer
`throughput_per_hour`	`n_served / (run_minutes / 60)`
`utilisation`	Total busy barista time divided by `n_baristas * finish_time`
`revenue`	Price of every completed order
`total_cost`	Ingredient cost of every completed order plus `n_baristas * barista_cost_per_hour * run_minutes / 60`
`profit`	`revenue - total_cost`

Waiting time means the time from arrival to the **start** of service. Lost customers pay nothing and use no ingredients. Baristas are paid for the planned `run_minutes`, even if the last customer finishes later.

If nobody is served, all waiting-time outputs must be `0.0`, not missing.

These counts must always balance:

```
n_arrivals = n_served + n_lost
n_lost = n_balked + n_reneged
n_served = n_served_simple + n_served_complex
```

`run_replications(config, seeds)` must return a pandas DataFrame with one row per seed, containing a `seed` column and every result field above.

The Readiness Check

You will receive:

- `cafe_model_template.py`, which shows the required structure

- `AI_USE_template.md`, the form for your AI Use Acknowledgment Statement
- `check_cafe_submission.py`, the same checker used for marking.

Put your completed `cafe_model.py` in your group folder. Then, from the repository root, run:

```
python mini_project/check_cafe_submission.py GROUP_01/cafe_model.py
```

Use your own group folder name instead of `GROUP_01`. The last line must be:

```
PASS: GROUP_01/cafe_model.py
```

All eight groups are checked with one command:

```
python mini_project/check_cafe_submission.py submissions/*/cafe_model.py
```

What the Checker Tests

The checker has two parts.

Contract checks. The checker makes sure that:

- the file loads and both functions exist
- the official configuration works without editing the file, and your code does not change the configuration dictionary
- every output is present, numeric, finite, and sensible
- the counts, revenue, cost, profit, and throughput all add up
- no served customer waited longer than the largest possible patience
- the same seed gives the same result, and a different seed changes it
- a café with no customers, a menu with only classic or only specialty drinks, `max_queue = 0`, and the `None` settings all behave correctly
- `run_replications` returns one correct row per seed.

Statistical checks. The checker then runs your `simulate_cafe` with 100 seeds in each of six different cafés: the standard café, one barista, a busy café with a short queue limit, very impatient customers, a bigger café with a different menu and prices, and a long quiet day. For each output, it compares your average over the 100 seeds with the average from the reference model:

```
z = (your mean - reference mean) / standard error.
```

Your numbers will not match the reference exactly, because your program uses the random numbers differently. That is fine. A correct model almost never gives $|z| > 5$, so any $|z| > 5$ is reported as a failure, with the café and the outputs that disagree. The café that fails usually tells you which rule to look at, for example the queue limit or reneging. The checker also tests that the number of arrivals varies from run to run as much as a Poisson process should.

The statistical checks run your model about 600 times. This usually takes less than a minute; the browser checker can be slower than the command line.

A file that prints `FAIL` is not ready. Fix every failure before the hackathon. A PASS is strong evidence that your model follows the rules, but it does not mark your notes, your experiments, or your explanation of the code.

Run the checker after every important change. Do not wait until the hackathon to try it for the first time.

Hackathon Day (Week 16)

First-hour rules: no AI, lab computers, no internet

The first hour is **AI Level 1 (No AI)**. It tests whether your group understands your own model well enough to change it.

- You work on the **lab computers, with no internet connection**. Python, Jupyter, NumPy, pandas, SciPy, and Matplotlib will be installed.
- You may use your group's own files (`cafe_model.py`, notebooks, and analysis scripts), your lab notebooks, the lecture notes, and Python's built-in help (`help(...)`, `?` and Shift+Tab in Jupyter).
- You may not use any AI tool, the internet, phones, or other personal devices. Do not talk to other groups.
- Bring your files and prepared slides on a USB drive (or as told in the briefing). In the Tuesday 17 November briefing, copy them to a lab computer and check that your model, your analysis script, your slides, and the command-line checker all work there with no internet.

Using AI or the internet in the first hour breaks the academic-integrity rules and is handled under the course academic-integrity policy. The second hour (pitches and questions) has no extra rules.

Get ready for the owner's question

You do not know the question in advance, so build a model that is easy to change. A group that prepares these can spend the hour on the question, not on rewriting code:

- an arrival generator whose rate can change during the day (for example a lunch rush), using thinning or separate time segments
- a way for a barista to be away for part of the day, or for an extra barista to work only part of the day
- one small function that chooses which waiting customer is served next, so that a priority rule is easy to add
- customer attributes you can extend (for example a customer type), with waiting times recorded by type
- an analysis script that runs several plans with the same 40 seeds, makes a comparison table, gives confidence intervals for the differences between plans, and draws a graph
- the first part of your pitch slides (see "Your pitch" below).

You may use AI to help prepare these at home, under the Level 3 rules above, but every member must understand them.

First hour: the owner's question

At the start of the Wednesday lab, each group receives a different challenge card on paper, with the seeds to use. In the first hour, your group must:

1. adapt a **copy** of your model (for example `challenge_model.py`) and keep `cafe_model.py` unchanged
2. compare all the plans on the card
3. use the seeds on the card
4. run at least 40 replications for each plan
5. make one comparison table and one graph

6. calculate a 95% confidence interval for the main outcome
7. make a practical recommendation
8. name one assumption or limitation that could change your recommendation.

After your pitch, hand in the following, in the way announced at the start of the session (for example on a USB drive or in a shared folder on the lab computer):

- your changed code
- the results table
- the graph
- your pitch slides.

Hand in the same files you used in your pitch.

Second hour: pitch to the owner

Each group has six minutes:

- four minutes to pitch
- two minutes for questions.

All three members must speak. One way to share it:

1. the model and the question
2. the experiment and the results
3. the recommendation and its limitation.

Your pitch: what to prepare

You may use as many slides as you like, but you only have four minutes, so keep them short. Prepare the first part at home before the hackathon. On the day, add the challenge slides at the end.

Before the hackathon (prepare at home):

- a title slide with your group number and members' names
- your café model: how a customer moves through the café (arrives, waits or balks, is served or reneges), the main rules, and the checks that convince you the model works.

On hackathon day (add at the end):

1. **The question:** the owner's problem in one sentence.
2. **What you changed:** the key change to your model, and any rule you had to decide yourselves.
3. **The experiment:** the plans you compared, with the same seeds and at least 40 runs each.
4. **The evidence:** your table and graph, with the main difference between plans and its 95% confidence interval. If the interval includes 0, say that the plans cannot be told apart; that is a valid result.
5. **The advice:** one sentence the owner can act on, for example "Hire the extra barista from 12:40 to 13:20."
6. **The catch:** one limitation that could change the advice.

Remember that the owner runs a café, not a statistics class. Give your numbers in baht, minutes and customers. Be ready for questions like "Why do you trust these numbers?" and "What would change your advice?"

Bring your prepared slides on the USB drive with your code, and check in the Tuesday briefing that they open on a lab computer without the internet.

Marking Guide

Part	Criterion	Marks
Prepared simulation (4)	Correct entities, queue, resources, events, and random inputs	1.5
	Reusable and configurable implementation	1.0
	Replications, seeds, outputs, and uncertainty analysis	1.0
	Verification, documentation, and assumptions	0.5
Live challenge (3)	Correct implementation of the assigned change	1.0
	Appropriate experiment using replications and uncertainty	1.0
	Evidence-based recommendation and meaningful limitation	1.0
Pitch communication (1)	Clear four-minute explanation supported by an effective table or graph	1.0
Individual understanding (1 per student)	Explains their contribution, relevant code, modelling choice, result, and answers questions without help	1.0

Marks are for modelling quality, reproducibility, experimental reasoning, and explanation. It is not a competition for the highest simulated profit: the best advice to the owner is the one your evidence supports.

Readiness-check rule. A PASS is compulsory but earns no marks by itself. If your submitted `cafe_model.py` shows FAIL, the mark for "Reusable and configurable implementation" (1.0) is 0. Other marks are given only for work that actually runs and that you can explain, and results that cannot be reproduced from your submitted code cannot earn experiment or recommendation marks.